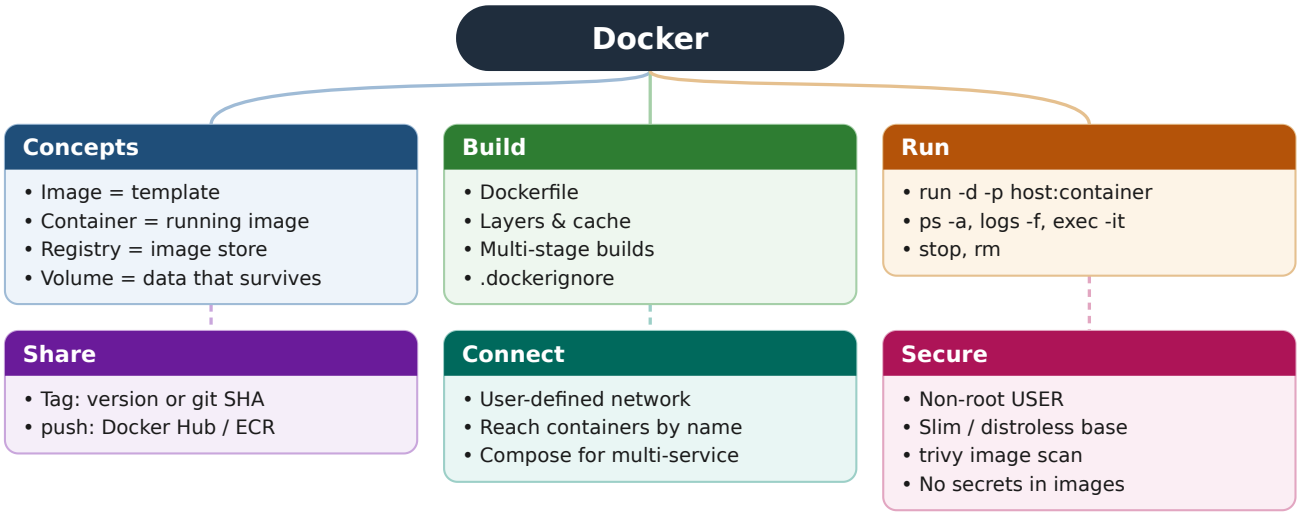
8. Docker

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 8 of 18 · Thirupathi Keloth · v2.0

Contents

8.0 Why containers
8.1 How containers work
8.2 Installing and verifying Docker
8.3 Images
8.4 Running containers
8.5 Dockerfile in depth
8.6 Multi-stage builds, BuildKit and tagging
8.7 Registries
8.8 Storage: volumes, bind mounts, tmpfs
8.9 Networking
8.10 Configuration and secrets
8.11 Docker Compose in depth
8.12 Resource limits, logging and restart behaviour
8.13 Container security
8.14 Troubleshooting
8.15 Docker in CI/CD
8.16 Docker vs orchestration
8.17 Lab: containerize a Python web app
8.18 Command reference
8.19 Interview questions and answers
8.20 Self-check: I can...



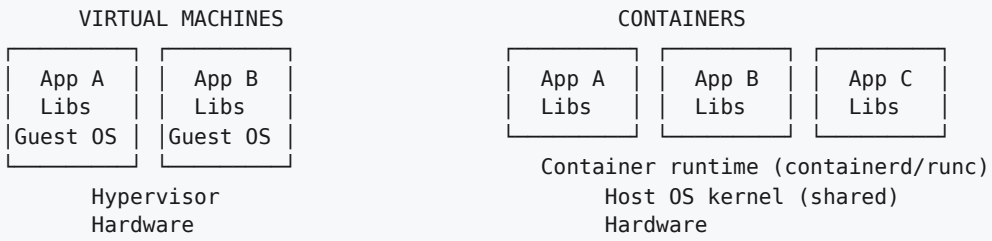
8.0 Why containers

The problem: "It works on my machine." Apps depend on specific language versions, libraries and OS packages; servers differ; deployments break. **The solution:** package the application **with everything it needs** into an **image**, and run it as an isolated **container** that behaves the same on a laptop, a CI runner, a VM or Kubernetes.

Where companies use containers: microservices, CI/CD build environments, Kubernetes/ECS/EKS deployments, local development stacks (Compose), batch jobs, serverless containers (Fargate, Cloud Run, Lambda images).

8.1 How containers work

Containers vs virtual machines



	VM	Container
Isolation	Separate kernel per VM (strong)	Shares the host kernel (process-level)
Size	GBs	MBs
Start time	Seconds–minutes	Milliseconds–seconds
Density	Tens per host	Hundreds per host
Best for	Different OSes, strong isolation	Packaging and running applications

In practice they're combined: containers run **inside** cloud VMs.

Linux building blocks

Feature	What it provides
Namespaces	Isolated *view* of the system: `pid` (own process tree; app is PID 1), `net` (own interfaces, IP, ports), `mnt` (own file system), `uts` (own hostname), `ipc`, `user` (UID mapping, rootless)
cgroups	*Limits* and accounting for CPU, memory, I/O, number of processes
Union / overlay file system (OverlayFS)	Read-only image layers stacked with a thin writable layer per container
Capabilities, seccomp, AppArmor/SELinux	Restrict what processes can do even as "root"

A container is just **a normal Linux process** with namespaces, cgroups and a layered file system. (On Windows/macOS, Docker Desktop runs a small Linux VM for this.)

Standards and runtimes

Component	Role
OCI (Open Container Initiative)	Standards for image format, runtime and distribution: images built by Docker run on containerd, CRI-O, Podman, Kubernetes
`runc`	Low-level runtime: creates namespaces/cgroups and starts the process
`containerd`	Manages images and container lifecycles (used by Docker and most Kubernetes clusters)
Docker Engine (`dockerd`)	Daemon providing the Docker API, builds (BuildKit), networking, volumes
Docker CLI (`docker`)	Client that talks to the daemon's API

Component	Role
Podman	Daemonless, rootless-friendly alternative with Docker-compatible commands
Registry	Stores and distributes images (Docker Hub, ECR, GHCR, Harbor)

```
docker CLI ──REST API──► dockerd ──► containerd ──► runc ──► container process
                            │
                            └── pulls/pushes images ◄──► registry
```

8.2 Installing and verifying Docker

Platform	How
Windows / macOS	**Docker Desktop** (on Windows enable the WSL2 backend)
Ubuntu server	Docker Engine from Docker's apt repository (`docker-ce` , `docker-ce-cli` , `containerd.io` , `docker-buildx-plugin` , `docker-compose-plugin`)
Amazon Linux 2023	`sudo dnf install -y docker && sudo systemctl enable --now docker`

```
docker version            # client + server versions (no server = daemon down / no permission)
docker info               # storage driver, cgroup version, number of containers/images
docker run --rm hello-world
sudo usermod -aG docker $USER   # run docker without sudo (log out/in after)
```

Security note: membership in the `docker` group is **equivalent to root** on that machine (you can mount the host file system into a container). Grant it only to trusted users; consider **rootless Docker** or Podman.

8.3 Images

Naming

```
registry.example.com:5000 / team / shop-api : 1.4.2
└──────── registry ────────┘ └ namespace ┘└ repo ┘ └ tag ┘
docker.io/library/nginx:1.27        (Docker Hub official image; "docker.io/library/" is implied)
123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2   (Amazon ECR)
ghcr.io/thirupathi/shop-api:sha-3f9a1c2                        (GitHub Container Registry)
nginx@sha256:4c0fdaa8b6...          (by DIGEST: immutable content hash)
```

- **Tag** = movable label (`latest` is just a default tag name, not "the newest").
- **Digest** = content hash; pinning by digest guarantees the exact image.

Layers

Each Dockerfile instruction that changes files creates a **read-only layer**. Layers are cached and shared between images (pulling a second image based on the same base is fast). A container adds one thin **writable layer** on top; it disappears when the container is removed.

Image commands

Command	Purpose
`docker pull nginx:1.27`	Download an image
`docker images` / `docker image ls`	List local images
`docker image inspect nginx:1.27`	Full metadata (layers, env, entrypoint, digest)
`docker history nginx:1.27`	Layers with sizes and the instructions that created them
`docker tag shop-api:1.4.2 ghcr.io/<user>/shop-api:1.4.2`	Add another name
`docker rmi image` / `docker image prune -a`	Remove one / all unused images
`docker save -o img.tar image` / `docker load -i img.tar`	Export/import as a file

Choosing a base image

Base	Size	Notes
`ubuntu`, `debian`	Larger	Full package manager and tools; easy debugging
`*-slim` (e.g. `python:3.12-slim`)	Medium-small	Debian with fewer packages: **good default**
`alpine`	Very small	Uses **musl** libc instead of glibc: some Python wheels/binaries misbehave or need compiling
distroless (`gcr.io/distroless/...`)	Tiny	No shell, no package manager: minimal attack surface; debug with ephemeral containers
`scratch`	Empty	For static binaries (Go, Rust)
Chainguard / hardened images	Small	Minimal, frequently patched images with few CVEs

Prefer **official** or verified images, pin versions (`python:3.12.7-slim`, not `python:latest`), and rebuild regularly to pick up security patches.

8.4 Running containers

docker run options

```
docker run -d --name web \
  -p 8080:80 \
  -e APP_ENV=prod --env-file ./app.env \
  -v web-data:/usr/share/nginx/html \
  --restart unless-stopped \
  --memory 256m --cpus 0.5 \
  --read-only --tmpfs /tmp --tmpfs /var/cache/nginx --tmpfs /var/run \
  nginx:1.27
```

Option	Meaning
`-d`	Detached (background)
`-it`	Interactive with a terminal (`docker run -it ubuntu bash`)

Option	Meaning
`--rm`	Delete the container when it exits
`--name web`	Name (otherwise random)
`-p 8080:80`	Publish **host port 8080** → container port 80 (`-p 127.0.0.1:8080:80` = local only)
`-P`	Publish all `EXPOSE`d ports to random host ports
`-e KEY=value` , `--env-file f`	Environment variables
`-v name:/path` / `-v /host/path:/path` / `--mount`	Named volume / bind mount
`--network mynet`	Attach to a network
`-w /app`	Working directory
`--user 1000:1000`	Run as a non-root UID/GID
`--entrypoint sh`	Override the image's ENTRYPOINT
`--restart no | on-failure | always | unless-stopped`	Restart policy
`--memory` , `--cpus` , `--pids-limit`	Resource limits
`--read-only` , `--cap-drop ALL` , `--security-opt no-new-privileges`	Hardening
`--init`	Small init process as PID 1 (signal handling, reaping zombies)
`--health-cmd` , `--health-interval`	Health check at run time

Lifecycle commands

Command	Purpose
`docker ps` / `docker ps -a`	Running / all containers (with exit codes)
`docker logs -f --tail 100 web`	Follow logs (stdout/stderr of the main process)
`docker exec -it web sh`	Shell inside a running container (`bash` if present)
`docker stop web`	SIGTERM, then SIGKILL after 10 s (`-t 30` to wait longer)
`docker kill web`	SIGKILL immediately
`docker start / restart web`	Start a stopped container / restart
`docker rm web` / `docker rm -f web`	Remove (force = stop first)
`docker inspect web`	Full JSON: state, IP, mounts, env, restart count, OOMKilled
`docker inspect -f '{{.State.Status}} {{.State.ExitCode}}' web`	Pick fields
`docker stats`	Live CPU/memory/network per container
`docker top web`	Processes inside

Command	Purpose
`docker cp web:/etc/nginx/nginx.conf .`	Copy files out/in
`docker port web`	Port mappings
`docker container prune`	Remove all stopped containers

Container states: created → running → (paused) → exited → removed. The container lives as long as its **main process (PID 1)** runs: if the command finishes, the container exits.

Avoid `docker commit` (snapshotting a modified container into an image): images must be reproducible from a Dockerfile.

8.5 Dockerfile in depth

Instructions

Instruction	Purpose	Notes
`FROM image:tag [AS name]`	Base image (and stage name)	First instruction (after optional `ARG`); pin versions
`ARG NAME=default`	Build-time variable	Not available at runtime; visible in image history (not for secrets)
`ENV KEY=value`	Runtime environment variable	Persisted in the image; defaults only, no secrets
`WORKDIR /app`	Working directory (created if missing)	Use instead of `RUN cd ...`
`COPY src dest`	Copy from the build context	`--chown=app:app`, `--from=build` (multi-stage)
`ADD src dest`	Like COPY + extracts tar archives + URLs	Prefer `COPY` unless you need tar extraction
`RUN command`	Execute at **build time**, creates a layer	Combine related commands, clean caches in the same RUN
`USER app`	User for following instructions and at runtime	Run as non-root
`EXPOSE 8000`	Documents the listening port	Doesn't publish anything
`VOLUME /data`	Marks a mount point	Data there isn't kept in image layers
`CMD [...]`	Default command / default arguments	Overridden by arguments to `docker run`
`ENTRYPOINT [...]`	Fixed executable	`CMD` becomes its default args
`HEALTHCHECK CMD ...`	Container health status	`--interval`, `--timeout`, `--retries`, `--start-period`
`LABEL key=value`	Metadata	OCI labels: `org.opencontainers.image.source`, `.version`, `.revision`
`STOPSIGNAL SIGTERM`	Signal sent by `docker stop`	
`SHELL ["/bin/bash", "-c"]`	Shell used by shell-form RUN	

Shell form vs exec form

```
CMD python app.py              # shell form: runs "/bin/sh -c 'python app.py'": sh is PID 1,
                               # SIGTERM goes to sh, app may not shut down gracefully
CMD ["python", "app.py"]       # exec form: python is PID 1 and receives signals → PREFERRED
```

CMD vs ENTRYPOINT

Dockerfile	`docker run image` runs	`docker run image --debug` runs
`CMD ["python", "app.py"]`	`python app.py`	`--debug` (replaces CMD!)
`ENTRYPOINT ["python", "app.py"]`	`python app.py`	`python app.py --debug`
`ENTRYPOINT ["python", "app.py"]` + `CMD ["--port", "8000"]`	`python app.py --port 8000`	`python app.py --debug`

Pattern: `ENTRYPOINT` = the program, `CMD` = default arguments. Entrypoint **scripts** (`docker-entrypoint.sh`) do setup then `exec "$@"` so the app becomes PID 1.

Build context and .dockerignore

`docker build -t app:1.0 .` sends the **build context** (the `.` folder) to the builder. `.dockerignore` excludes files: smaller/faster builds, no secrets or junk in images.

```
.git
.venv
__pycache__
*.pyc
node_modules
.env
*.pem
Dockerfile*
docker-compose*.yml
tests/
docs/
```

Layer caching rules

The builder reuses a cached layer if the instruction **and** its inputs didn't change; once one layer changes, **all following layers rebuild**. Therefore: put things that change rarely first (base, OS packages, dependency files), and your source code **last**.

```
# GOOD: dependencies cached until requirements.txt changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# BAD: any code change reinstalls all dependencies
# COPY . .
# RUN pip install -r requirements.txt
```

Production Dockerfile: Python web app

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*

RUN useradd --create-home --uid 10001 appuser

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY --chown=appuser:appuser . .

USER 10001
EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD ["curl", "-fsS", "http://localhost:8000/health"]

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", "app:app"]
```

Key points: pinned slim base · `PYTHONUNBUFFERED` so logs appear immediately · OS packages installed and apt lists removed **in the same layer** · non-root user with a **numeric UID** (lets Kubernetes verify `runAsNonRoot`) · dependencies before code · exec-form CMD · app binds to **`0.0.0.0`** (binding to `127.0.0.1` inside a container makes it unreachable from outside). Linters such as hadolint also suggest pinning apt package versions (`curl=8.5.0-2`); strict teams do, others rely on pinned base images + regular rebuilds.

Node.js and Java examples

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
USER 1000
EXPOSE 3000
CMD ["node", "server.js"]
```

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/app.jar app.jar
RUN useradd -r -u 10001 app
USER 10001
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

`-XX:MaxRAMPercentage` makes the JVM respect the container's memory limit.

8.6 Multi-stage builds, BuildKit and tagging

Multi-stage build

```
# syntax=docker/dockerfile:1
FROM golang:1.23 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o /out/api ./cmd/api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/api /api
USER 65532:65532
EXPOSE 8080
ENTRYPOINT ["/api"]
```

Compilers, source code and build tools stay in the first stage; the final image contains **only the binary** (a few MB, almost no CVEs). Python/Node apps use the same idea: build wheels or bundles in a "builder" stage, copy only results.

BuildKit features

```
# Cache package downloads between builds (not stored in the image)
RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt

# Use a secret during build WITHOUT leaving it in any layer
RUN --mount=type=secret,id=npm_token \
    NPM_TOKEN=$(cat /run/secrets/npm_token) npm ci
```

```
docker build --secret id=npm_token,env=NPM_TOKEN -t app:1.0 .
docker buildx build --platform linux/amd64,linux/arm64 -t ghcr.io/<user>/app:1.0 --push .
docker build --target build -t app:builder .        # build only up to a named stage
docker build --no-cache --pull -t app:1.0 .         # ignore cache, refresh base image
```

Never pass secrets with `ARG` / `ENV` or `COPY` them in: they end up in the image history.

Tagging strategy

Tag	Use
`1.4.2`	Release version (semantic)
`sha-3f9a1c2` (git commit)	Exact traceability from image to code; CI default
`1.4`, `1`	Floating "latest patch / latest minor" (convenience)
`latest`	Avoid in deployments (ambiguous); OK for local use

Deploy the **same immutable image** to dev → staging → prod (promote, don't rebuild). Turn on **tag immutability** in registries.

8.7 Registries

Registry	Notes
Docker Hub	Default public registry; **pull rate limits** for anonymous/free users: authenticate or mirror
Amazon ECR	Private per AWS account/region; IAM auth; scanning; lifecycle policies
GitHub Container Registry	`ghcr.io`; auth with `GITHUB_TOKEN` in Actions
Google Artifact Registry, Azure ACR	Other clouds
Harbor / Nexus / Artifactory	Self-hosted, with scanning and replication

```
docker login                                        # Docker Hub
echo "$GHCR_TOKEN" | docker login ghcr.io -u <user> --password-stdin
aws ecr get-login-password --region ap-south-1 \
  | docker login --username AWS --password-stdin 123456789012.dkr.ecr.ap-south-1.amazonaws.com

docker tag shop-api:1.4.2 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
docker push 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
docker logout ghcr.io
```

`--password-stdin` keeps tokens out of shell history and process lists. Credentials are stored in `~/.docker/config.json` (use a credential helper).

8.8 Storage: volumes, bind mounts, tmpfs

```
container writable layer   → deleted with the container (don't keep data here)
named volume  (-v pgdata:/var/lib/postgresql/data)  → managed by Docker in /var/lib/docker/volumes
bind mount    (-v $(pwd)/src:/app)                  → a host directory/file mounted in
tmpfs         (--tmpfs /tmp)                        → in memory, gone at stop
```

Type	Best for	Watch out
Named volume	Databases and persistent app data	Back it up; survives `docker rm` (removed with `docker volume rm` / `compose down -v`)
Bind mount	Development (live code), config files from host	Host path must exist; UID/permission mismatches; ties container to one host
tmpfs	Secrets/temp data that shouldn't touch disk	Lost on stop; uses RAM

```
docker volume create pgdata
docker volume ls
docker volume inspect pgdata
docker run --rm -v pgdata:/data -v "$(pwd)":/backup alpine \
  tar -czf /backup/pgdata-backup.tgz -C /data .          # back up a volume
docker volume prune                                       # remove unused volumes (careful!)
```

Permission problems ("permission denied" writing to a mount): the container user's UID doesn't match the owner of the host directory. Fix by `chown`-ing the host path to that UID, running with `--user $(id -u):$(id -g)`, or using named volumes.

8.9 Networking

Driver	Behaviour	Use
`bridge` (default `docker0`)	Private network on the host; NAT to the outside	Single containers
User-defined bridge	Like bridge + **automatic DNS by container name** + better isolation	Multi-container apps on one host (Compose creates one)
`host`	Container uses the host network stack directly (no `-p`, no isolation)	Performance-sensitive tools, Linux only
`none`	No network	Batch jobs that must be offline
`overlay`	Spans multiple hosts	Docker Swarm
`macvlan` / `ipvlan`	Container gets its own address on the physical network	Legacy apps needing a LAN IP

```
docker network create shopnet
docker run -d --name db --network shopnet --env-file db.env \
  -v pgdata:/var/lib/postgresql/data postgres:16   # db.env (git-ignored): POSTGRES_PASSWORD
docker run -d --name api --network shopnet -p 8000:8000 -e DB_HOST=db shop-api:1.4.2
docker network inspect shopnet            # connected containers and their IPs
docker network connect othernet api       # attach to a second network
docker exec api getent hosts db           # name resolution via Docker's embedded DNS (127.0.0.11)
```

Port publishing: `-p 8080:80` creates NAT rules (iptables) from host:8080 to container:80. `-p 127.0.0.1:8080:80` exposes only on the host's loopback (safer for admin UIs/databases). Note that Docker's iptables rules can bypass `ufw`: don't publish database ports publicly by accident.

Remember: `localhost` inside a container = the container itself. To reach another container use its **name** on a shared network; to reach the host use `host.docker.internal` (Docker Desktop, or `--add-host=host.docker.internal:host-gateway` on Linux).

8.10 Configuration and secrets

Need	Approach
Non-secret config	Environment variables (`-e`, `--env-file`, Compose `environment`), or a mounted config file
Secrets in development	`.env` file **excluded from Git**, loaded by Compose
Secrets in production	Orchestrator secrets injected at runtime: Kubernetes Secrets (+ External Secrets), ECS `secrets` from Secrets Manager/SSM, Docker/Compose secrets mounted as files in `/run/secrets/`
Build-time secrets	BuildKit `--mount=type=secret`

Twelve-factor app principle: store config in the environment; the same image runs everywhere with different config.

Prefer **secret files** (`*_FILE` variables, many official images support them, e.g. `POSTGRES_PASSWORD_FILE`) over plain environment variables, because env vars show up in `docker inspect`, process listings and crash dumps.

8.11 Docker Compose in depth

Compose defines and runs multi-container applications from one YAML file (`compose.yaml`). Used for local development, integration tests in CI, and small single-host deployments.

Full example: web app + PostgreSQL + Redis + nginx

```
# compose.yaml
name: shop

services:
  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    image: ghcr.io/thirupathi/shop-api:${API_TAG:-dev}
    environment:
      APP_ENV: ${APP_ENV:-dev}
      DB_HOST: db
      DB_NAME: shop
      DB_USER: shop
      REDIS_URL: redis://cache:6379/0
    env_file:
      - .env                          # secrets like DB_PASSWORD, not committed
    depends_on:
      db:
        condition: service_healthy    # wait for the DB health check, not just "started"
      cache:
        condition: service_started
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost:8000/health"]
      interval: 15s
      timeout: 3s
      retries: 5
      start_period: 20s
    restart: unless-stopped
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
    networks: [backend, frontend]

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: shop
      POSTGRES_PASSWORD: ${DB_PASSWORD:?DB_PASSWORD must be set in .env}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shop -d shop"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped
    networks: [backend]

  cache:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redisdata:/data
    networks: [backend]

  proxy:
    image: nginx:1.27-alpine
    ports:
      - "80:80"
```

```
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on: [api]
    networks: [frontend]

  adminer:
    image: adminer
    ports:
      - "127.0.0.1:8081:8080"        # local-only admin UI
    profiles: [debug]                # only started with --profile debug
    networks: [backend]

volumes:
  pgdata:
  redisdata:

networks:
  frontend:
  backend:
```

Key	Meaning
`build` / `image`	Build from a Dockerfile and/or the image name to use or tag
`environment`, `env_file`	Variables (`${VAR:-default}` and `${VAR:?error}` interpolate from the shell or a `.env` file next to compose.yaml)
`ports`	`"host:container"`; quote them (YAML can misread `22:22`-style values)
`volumes`	Named volumes (declared at the bottom) or bind mounts (`./path:/path:ro`)
`depends_on` + `condition: service_healthy`	Start order **and** readiness
`healthcheck`	Health status shown in `docker compose ps`
`restart`	Restart policy
`deploy.resources.limits`	CPU/memory limits
`networks`	Isolation: `proxy` can't reach `db` directly here
`profiles`	Optional services
`command` / `entrypoint`	Override the image defaults

Compose commands

Command	Purpose
`docker compose up -d`	Create/start everything in the background
`docker compose up -d --build`	Rebuild images first
`docker compose ps`	Status and health
`docker compose logs -f api`	Follow one service's logs
`docker compose exec api sh`	Shell in a running service
`docker compose run --rm api pytest`	One-off command in a new container
`docker compose restart api` / `stop` / `start`	Control services

Command	Purpose
`docker compose down`	Stop and remove containers and networks
`docker compose down -v`	...and **delete volumes** (data loss!)
`docker compose config`	Show the final merged configuration (validate files)
`docker compose pull` / `build`	Update images / build
`docker compose up -d --scale api=3`	Several replicas (without fixed host ports)
`docker compose --profile debug up -d`	Include profile services
`docker compose -f compose.yaml -f compose.prod.yaml up -d`	Layer override files

Override files: `compose.override.yaml` is applied automatically (dev settings like bind mounts and debug ports); production uses explicit `-f` files.

8.12 Resource limits, logging and restart behaviour

Limits

Limit	Option	When exceeded
Memory	`--memory 512m`	Kernel OOM-kills the process → exit code **137**, `OOMKilled: true`
CPU	`--cpus 1.5`	Throttled (slower), not killed
Processes	`--pids-limit 200`	Can't fork more (protects against fork bombs)
Open files	`--ulimit nofile=65536:65536`	"Too many open files"

Languages must respect container limits: modern JVMs do (`MaxRAMPercentage`), Node may need `--max-old-space-size`, Python worker counts should match CPU limits.

Logging

Containers should log to **stdout/stderr**; Docker captures it with a **logging driver**. Default `json-file` grows forever unless you rotate it:

```
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
```

(`/etc/docker/daemon.json`, then `sudo systemctl restart docker`.) Other drivers: `local`, `journald`, `awslogs` (CloudWatch), `fluentd`, `gelf`. In Kubernetes, node agents (Fluent Bit, Alloy) collect container logs.

Restart policies

`no` (default) · `on-failure[:max]` (restart on non-zero exit) · `always` (also after daemon restart) · `unless-stopped` (like always unless you stopped it). For real high availability use an orchestrator (Kubernetes, ECS), not restart policies.

8.13 Container security

Practice	How
Minimal, trusted, pinned base images	slim/distroless/hardened images; pin version (or digest); rebuild regularly
Run as non-root	`USER` in Dockerfile; `--user`; Kubernetes `runAsNonRoot`
Least privileges	`--cap-drop ALL` (add back only what's needed, e.g. `NET_BIND_SERVICE`), `--security-opt no-new-privileges`
Never `--privileged`	Gives the container nearly full host access
Never mount `/var/run/docker.sock` into untrusted containers	Equals root on the host
Read-only root file system	`--read-only` + `tmpfs` for writable paths
No secrets in images	BuildKit secrets, runtime secret injection; scan images for secrets
Scan images	`trivy image app:1.0`, Docker Scout, Grype, ECR/Inspector scanning; fail CI on CRITICAL
SBOM	`syft app:1.0` / `trivy image --format cyclonedx` lists everything inside
Sign and verify images	`cosign sign` / `cosign verify`; enforce in Kubernetes admission policies
Limit resources	Memory/CPU/PIDs limits prevent noisy neighbours and DoS
Kernel protections	Default seccomp and AppArmor/SELinux profiles stay enabled
Rootless Docker / Podman	Daemon and containers without root on the host
Lint Dockerfiles	`hadolint Dockerfile`
Benchmarks	CIS Docker Benchmark (`docker-bench-security`)

```
trivy image --severity HIGH,CRITICAL --exit-code 1 shop-api:1.4.2   # fail if serious CVEs
trivy fs --scanners vuln,secret,misconfig .                         # scan source/Dockerfiles
```

8.14 Troubleshooting

Exit codes

Exit code	Meaning	Typical cause
0	Process finished successfully	Command isn't a long-running server (container "exits immediately" by design)
1	Application error	Read `docker logs`
126	Command not executable	Missing `chmod +x` on entrypoint script
127	Command not found	Wrong CMD/ENTRYPOINT path, binary missing in the image (e.g. `bash` in Alpine/distroless)
137	Killed (SIGKILL)	**OOM** (`docker inspect` → `OOMKilled: true`) or `docker kill`

Exit code	Meaning	Typical cause
139	Segmentation fault	Native crash, architecture/library mismatch
143	SIGTERM	Normal `docker stop`

Common problems

Symptom	Investigation / fix
Container exits immediately	`docker ps -a` (exit code) → `docker logs` → run interactively: `docker run -it --entrypoint sh image`
Restart loop	Logs from the previous run (`docker logs`), missing env/secret, dependency not ready (add health checks/retries), OOM
App unreachable from host	App listening on `127.0.0.1` inside the container (must be `0.0.0.0`) · missing/wrong `-p` · firewall · wrong port
`port is already allocated`	Another container/process uses the host port (`docker ps`, `ss -tulpn`)
Containers can't reach each other	Not on the same user-defined network · using `localhost` · wrong service name/port
`exec format error`	Image built for another CPU architecture (arm64 vs amd64) → `--platform` / buildx multi-arch
`permission denied` on volume	UID mismatch between container user and host files
`no space left on device`	`docker system df` → `docker system prune` (careful with `-a --volumes`), log rotation, remove old images
Build is slow / always rebuilds	Cache busted early (COPY . before deps), huge build context (missing `.dockerignore`)
`toomanyrequests` pulling images	Docker Hub rate limit → log in, use a mirror/ECR pull-through cache
`Cannot connect to the Docker daemon`	Daemon not running (`systemctl status docker`) or user not in `docker` group
Debugging a distroless image	`docker debug` (Docker Desktop) or run a debug sidecar sharing the namespace: `docker run -it --pid=container:app --net=container:app busybox`

Diagnostic commands

```
docker ps -a
docker logs --tail 200 -f app
docker inspect app --format '{{json .State}}' | jq
docker exec -it app sh -c 'env | sort; ss -tulpn 2>/dev/null || netstat -tulpn'
docker events --since 10m             # what the daemon did (starts, dies, OOM)
docker system df                      # disk usage by images/containers/volumes/cache
```

8.15 Docker in CI/CD

```
# GitHub Actions excerpt: build, scan, push
- uses: docker/setup-buildx-action@v3
- uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}
- uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: ghcr.io/${{ github.repository }}:sha-${{ github.sha }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

Topic	Practice
Tags	Commit SHA for every build, semantic version on release tags
Cache	BuildKit cache (registry or CI cache) for fast builds
Scan	Trivy/Scout step that fails on critical vulnerabilities
Build without a Docker daemon	Kaniko, Buildah, BuildKit rootless (common inside Kubernetes runners)
Docker-in-Docker vs socket mounting	Both have security trade-offs; prefer rootless builders or managed runners
Promotion	Build once, push, deploy the same digest to every environment

8.16 Docker vs orchestration

Docker runs containers on **one host**. Production systems need scheduling across many hosts, self-healing, scaling, rolling updates, service discovery and secrets: that's **orchestration**: **Kubernetes** (industry standard), **Amazon ECS**, Docker Swarm (simple, less used), Nomad. Compose is for local/dev and small single-host setups.

8.17 Lab: containerize a Python web app

```
# api/app.py
import os
from flask import Flask, jsonify

app = Flask(__name__)


@app.get("/health")
def health():
    return jsonify(status="ok")


@app.get("/")
def index():
    return jsonify(message="Hello from shop-api", env=os.getenv("APP_ENV", "dev"))
```

```
# api/requirements.txt
flask==3.0.3
gunicorn==23.0.0
```

1. Write the Dockerfile from 8.5 (Python) in `api/`, and a `.dockerignore`.
2. `docker build -t shop-api:1.0 api/` → `docker images shop-api` (note the size).
3. `docker run -d --name api -p 8000:8000 -e APP_ENV=lab shop-api:1.0` → `curl localhost:8000/` and `/health`.
4. `docker exec -it api sh -c 'id; ps aux'` → confirm it runs as `appuser`, not root.
5. Change a line in `app.py`, rebuild: only the last layers rebuild (cache).
6. `trivy image shop-api:1.0` → read the findings; try a different base image and compare.
7. Write a `compose.yaml` with the API + PostgreSQL (health check, volume, `.env` for the password) → `docker compose up -d` → `docker compose ps`.
8. Stop the DB container and watch the API's behaviour; read logs; start it again.
9. **Cleanup:** `docker compose down -v`, `docker rm -f api`, `docker image prune`.

8.18 Command reference

Area	Commands
Images	`pull`, `images`, `build -t`, `tag`, `push`, `rmi`, `history`, `image inspect`, `image prune`
Containers	`run`, `ps -a`, `logs -f`, `exec -it`, `stop`, `start`, `restart`, `rm -f`, `inspect`, `stats`, `top`, `cp`, `container prune`
Volumes	`volume create/ls/inspect/rm/prune`
Networks	`network create/ls/inspect/connect/rm`
Compose	`compose up -d --build`, `ps`, `logs -f`, `exec`, `run --rm`, `down [-v]`, `config`
Build	`buildx build --platform ... --push`, `build --target`, `--secret`, `--no-cache`
System	`version`, `info`, `system df`, `system prune`, `events`, `login/logout`

8.19 Interview questions and answers

#	Question	Answer
1	Container vs VM?	Containers share the host kernel and isolate processes (namespaces/cgroups); VMs virtualize hardware with separate kernels
2	Image vs container?	Image = read-only layered template; container = running instance with a writable layer
3	What are namespaces and cgroups?	Namespaces isolate what a process sees (PIDs, network, mounts…); cgroups limit resources
4	What is a Docker layer and how does caching work?	Each instruction creates a layer; unchanged instructions reuse cache until the first changed one
5	CMD vs ENTRYPOINT?	ENTRYPOINT is the fixed executable; CMD gives default args (or the default command), easily overridden
6	COPY vs ADD?	COPY copies files; ADD also extracts tars and fetches URLs. Prefer COPY

#	Question	Answer
7	Shell form vs exec form?	Exec form runs the process as PID 1 so it receives signals; shell form wraps it in `sh -c`
8	How do you reduce image size?	Slim/distroless bases, multi-stage builds, `.dockerignore`, clean caches in the same RUN, fewer dependencies
9	What is a multi-stage build?	Several FROM stages; build tools in early stages, only artifacts copied to the final image
10	Volume vs bind mount?	Docker-managed storage for persistent data vs a host path mounted into the container
11	How do containers communicate?	On a user-defined network by container/service name via embedded DNS
12	What does `-p 8080:80` do?	Maps host port 8080 to container port 80
13	Why does my app work in the container but not from the host?	It's bound to 127.0.0.1 inside the container, or the port isn't published
14	Exit code 137?	SIGKILL, usually OOM because of the memory limit
15	How do you handle secrets in Docker?	Never bake into images; inject at runtime (orchestrator secrets, files in /run/secrets), BuildKit secrets for builds
16	How do you secure containers?	Non-root, minimal images, scanning, drop capabilities, read-only FS, no privileged/docker.sock, resource limits, signing
17	What is Docker Compose for?	Defining and running multi-container apps on one host (dev, testing, small deployments)
18	`depends_on` guarantees the DB is ready?	Only start order; use `condition: service_healthy` with a health check (and retries in the app)
19	Why not use the `latest` tag in production?	It's mutable and ambiguous; use version/SHA tags or digests for reproducibility and rollback
20	How do you debug a container that keeps restarting?	Exit code + `docker logs` + `docker inspect` (OOMKilled, env) + run it interactively with a shell entrypoint
21	What is OCI?	Open standards for container images and runtimes so images work across tools (Docker, containerd, Podman, Kubernetes)
22	Docker vs Kubernetes?	Docker builds/runs containers on a host; Kubernetes orchestrates containers across a cluster

8.20 Self-check: I can…

- [ ] Explain containers vs VMs, namespaces, cgroups, overlay file systems and OCI/containerd/runc.
- [ ] Install Docker and explain the security impact of the `docker` group.
- [ ] Name, pull, inspect, tag and remove images; choose a suitable base image.
- [ ] Run containers with ports, env, volumes, limits, restart policies and hardening options.
- [ ] Write production Dockerfiles: caching order, exec form, non-root, health checks, `.dockerignore`.
- [ ] Explain CMD vs ENTRYPOINT behaviour precisely.
- [ ] Build multi-stage and multi-architecture images; use BuildKit cache and secrets.
- [ ] Tag and push images to Docker Hub, GHCR and ECR.
- [ ] Use volumes, bind mounts and tmpfs correctly and back up a volume.
- [ ] Configure container networking and explain port publishing and container DNS.
- [ ] Write a full Compose stack with health checks, profiles, overrides and limits.

- [ ] Configure log rotation and understand OOM behaviour.
- [ ] Harden and scan images; generate an SBOM.
- [ ] Troubleshoot by exit code, logs, inspect, events and disk usage.
- [ ] Build and push images in CI with caching and scanning.

REMEMBER

- Port mapping is **-p HOST:CONTAINER**.
- Dockerfile order: **base → dependencies → code** (code changes most, so it goes last).
- Exit code **137** = killed (usually out of memory). Exit **1** = application error → read the logs.
- Inside a container, `localhost` means the container itself.